

Integrated ATM Cash Replenishment and Vehicle Routing Optimization

Kolkata Case Study Using Synthetic Data OR Study by Nikhil Vatsa

Abstract

This project develops an Operations Research model for the replenishment of ATMs by a single cash truck in Kolkata. Unlike a basic shortest-route problem, the proposed model jointly considers ATM selection, cash allocation, truck capacity, demand uncertainty, ATM priority, service time, time windows, and vehicle routing. Three demand scenarios: low, normal and high are used to represent uncertainty. The optimization is formulated as a mixed-integer linear programming (MILP) model and implemented using the Gurobi Optimizer. All ATM data, demand figures, and operating parameters are synthetic and are intended only for academic demonstration.

Introduction

Cash replenishment is a recurring logistics challenge for banks. When an ATM receives too little cash, it may run out and become unavailable to customers, which harms service quality and customer satisfaction. When too much cash is loaded, the bank incurs additional cash-handling costs and opportunity costs, since idle cash in machines could be used elsewhere. These trade-offs are further complicated by the operational constraints of cash-in-transit services, including vehicle capacity, working hours, travel distance, and service requirements.

This study addresses the following operations research question: how should a bank decide which ATMs to visit, how much cash to deliver to each, and in what order to visit them, given limited truck capacity and uncertain ATM cash demand? The topic is of both practical and personal interest to the author. Prior professional experience in the banking sector provided direct insight into the operational importance of ATM cash management, and from the customer's perspective, an empty ATM is a common and frustrating experience. By identifying an efficient replenishment strategy, this study aims to help banks improve ATM availability and, consequently, customer satisfaction. The problem is also computationally challenging, as its complexity grows rapidly with the number of ATMs, since the number of possible visit sequences and delivery quantities expands combinatorially.

The objective of the study is to minimize the total cost incurred by the bank in the replenishment operation, subject to the operational constraints described above.

Problem Description

In the current problem, we are given 15 ATMs located in different parts of Kolkata. The objective is to select a subset of ATMs to replenish, determine how much cash to deliver to each selected ATM, and optimize the route taken by the truck. The truck has a capacity limit of 25 lakh INR, reflecting physical and security requirements, and a maximum operating time of 8 hours, from 9 AM to 5 PM. All transportation and servicing must be completed within this window, and the truck must return to its base location. Only 10 ATMs can be visited in a single route.

The problem is set around a fictional cash depot located at 22.5726°N, 88.3639°E. The data used in this study is synthetic and serves only to formulate the solution approach. Given accurate data, the same model can be applied to solve the corresponding real-world scenario. Each ATM has an assigned priority level, and all priority 3 ATMs must be visited. Servicing time varies by ATM, ranging from 10 to 13 minutes. The average truck speed is assumed to be 25 km/h, and road distances are calculated as 1.25 times the straight-line haversine distance.

ATMs are prioritized according to their location and historical information. Because the actual demand at each ATM cannot be predicted exactly, the model also accounts for demand uncertainty, meaning that the forecasted demand may differ from the actual demand. Forecast demand data is available for each ATM, and three demand scenarios are considered: low demand at 0.80 times the forecast with a probability of 0.20, normal demand at 1.00 times the forecast with a probability of 0.50, and high demand at 1.30 times the forecast with a probability of 0.30. These probabilities are subjective and can be revised based on feedback. Since the exact demand is unknown, the model works with the expected value of demand across these scenarios.

Combining all of this information, the study determines the optimal strategy for the truck on a given day. The model minimizes the total of travel costs, visiting costs, cash holding costs, and expected ATM shortage costs. Because demand is uncertain, cash shortages are penalized in the objective so that the model actively seeks to keep them to a minimum.

Model Formulation

Decision variables:

$x_{ij} = 1$ if the truck travels directly from node i to node j ; 0 otherwise

$y_i = 1$ if ATM i is visited; 0 otherwise

$q_i =$ cash delivered to ATM i measured in lakh rupees

$s_i^k =$ shortage at ATM i under demand scenario k

$t_i =$ arrival time at ATM i

u_i = route – order variable used to eliminate subtours

Objective function:

Min Travel Cost + Visit Cost + Cash Holding Cost + Expected Shortage Cost

Travel Cost = Travel cost per km * $\sum \sum dist_{ij} * x_{ij}$

Visit Cost = Visit fixed cost * $\sum y_i$

Cash Holding Cost = cash holding cost per lakh rupees * $\sum q_i$

Expected Shortage Cost = shortage penalty * $\sum \sum probability^k * s_i^k$

Travel cost depends on the total distance travelled. A small fixed visit cost discourages unnecessary stops. Cash holding cost represents the disadvantage of carrying and loading additional cash. Shortage cost is deliberately large so that the model values ATM service.

This objective creates a meaningful trade-off: visiting more ATMs can reduce shortage, but it increases routing cost and consumes limited truck capacity.

Constraints:

Flow constraints: a visited ATM has exactly one incoming and one outgoing route.

$$\sum_j x_{ji} = y_i$$

$$\sum_j x_{ij} = y_i$$

Depot constraints: the truck leaves and returns to the depot exactly once.

$$\sum_j x_{0j} = 1$$

$$\sum_i x_{i0} = 1$$

Arc constraints: the truck cannot have an arc between two unvisited ATMs.

$$x_{ij} \leq y_i$$

$$x_{ij} \leq y_j$$

Visit limit: no more than ten ATMs can be visited.

$$\sum_i y_i \leq 10$$

Priority constraint: every priority-3 ATM must be visited.

$$y_i = 1 \text{ if atm has priority} = 3$$

MTZ constraints eliminate disconnected subtours.

$$u_i \leq (N - 1) * y_i$$

$$u_i \geq y_i$$

$$u_i - u_j + (N - 1) * x_{ij} \leq N - 2$$

Truck capacity: total cash delivered cannot exceed ₹25 lakh.

$$\sum_i q_i \leq 25$$

ATM capacity: cash delivered cannot exceed the remaining capacity of the ATM.

$$q_i \leq \text{max delivery}_i * y_i$$

Shortage constraints: scenario demand can exceed available cash, but the resulting shortage is penalized.

$$\text{current}_i + q_i + s_i^k \geq \text{demand}_i$$

Route duration: travel time plus ATM service time cannot exceed eight hours.

$$\text{travel time} + \text{service time} \leq \text{max route hours}$$

$$\text{travel time} = \sum_i \sum_j (\text{dist}_{ij} / \text{average speed}) * x_{ij}$$

$$\text{service time} = \sum_i \text{service hour}_i * y_i$$

Time constraint: if truck travels from i to j arrival time at j must be greater than the service time at i and the time to travel between i and j

$$t_j \geq t_i + \text{service hour}_i + \text{dist}_{ij} / \text{average speed} - M(1 - x_{ij})$$

Time windows: arrival times must remain between 09:00 and 17:00.

$$t_j \geq 9 + \text{dist}_{0j} / \text{average speed} - M(1 - x_{0j})$$

*M = a large number (50 in this case)

Method

The accompanying [Python program](#) uses gurobipy. Binary variables represent route and visit decisions, while continuous variables represent cash allocation, shortage, arrival time, and route order. Gurobi minimizes the integrated objective subject to the constraints.

The program also prints the selected ATMs, optimized route, total distance, cash loaded, route duration, cash allocation, and shortage under each demand scenario.

The python program starts with importing of required libraries for this project. We also create a distance function to calculate the distance between two locations using the haversine distance formula. We also define some constants.

We start off by reading the csv file provided as a pandas dataframe. We convert this dataframe into a dictionary and add the central depot location as it is not provided in the file. Lastly before proceeding with our model, we create a distance matrix between each provided location in the dictionary. This distance matrix will be used in the model.

We first start by declaring the variables as described in the previous section. Variables like x_{ij} and y_i are binary variables. All the rest are continuous variables. For continuous variables we also mention the lower and upper bound of the variable.

```
model = gp.Model("Integrated_ATM_Cash_Routing")
```

```
#  $x_{[i,j]}$  = 1 if truck travels  $i \rightarrow j$ 
```

```
x = model.addVars(
```

```
[(i, j) for i in nodes for j in nodes if i != j],
```

```
vtype=GRB.BINARY,
```

```
name="x"
```

```
)
```

```
#  $y_{[i]}$  = 1 if ATM  $i$  is visited
```

```
y = model.addVars(customers, vtype=GRB.BINARY, name="y")
```

```
#  $q_{[i]}$  = cash delivered to ATM  $i$  (lakh INR)
```

```
q = model.addVars(  
    customers,  
    lb=0,  
    ub=GRB.INFINITY,  
    vtype=GRB.CONTINUOUS,  
    name="q"  
)
```

```
# shortage[i,s] = shortage in scenario s
```

```
shortage = model.addVars(  
    customers,  
    list(SCENARIOS.keys()),  
    lb=0,  
    vtype=GRB.CONTINUOUS,  
    name="shortage"  
)
```

```
# t[i] = arrival time at ATM i
```

```
t = model.addVars(  
    customers,  
    lb=9.0,  
    ub=17.0,  
    vtype=GRB.CONTINUOUS,  
    name="arrival"  
)
```

```
# u[i] = MTZ ordering variable
```

```

u = model.addVars(
    customers,
    lb=0,
    ub=N-1,
    vtype=GRB.CONTINUOUS,
    name="order"
)

```

The constraints are included in the model as mentioned above.

```

# -----
# Routing constraints
# -----

```

If ATM i is visited, exactly one incoming arc.

for i in customers:

```

    model.addConstr(
        gp.quicksum(x[j, i] for j in nodes if j != i) == y[i],
        name=f"one_in_{i}"
    )

```

If ATM i is visited, exactly one outgoing arc.

for i in customers:

```

    model.addConstr(
        gp.quicksum(x[i, j] for j in nodes if j != i) == y[i],
        name=f"one_out_{i}"
    )

```

Truck leaves and returns to depot once.

```

model.addConstr(
    gp.quicksum(x[0, j] for j in customers) == 1,
    name="depot_departure"
)

model.addConstr(
    gp.quicksum(x[i, 0] for i in customers) == 1,
    name="depot_return"
)

# No arc between two unvisited ATMs.
for i in customers:
    for j in customers:
        if i != j:
            model.addConstr(x[i, j] <= y[i], name=f"arc_from_{i}_{j}")
            model.addConstr(x[i, j] <= y[j], name=f"arc_to_{i}_{j}")

# At most MAX_VISITS ATMs.
model.addConstr(
    gp.quicksum(y[i] for i in customers) <= MAX_VISITS,
    name="maximum_visits"
)

# Priority-3 ATMs must be visited.
for i in customers:
    if int(df.iloc[i-1]["priority"]) == 3:
        model.addConstr(y[i] == 1, name=f"priority_{i}")

```

```
# MTZ subtour elimination.
```

```
for i in customers:
```

```
    model.addConstr(u[i] <= (N-1) * y[i], name=f"order_upper_{i}")
```

```
    model.addConstr(u[i] >= y[i], name=f"order_lower_{i}")
```

```
for i in customers:
```

```
    for j in customers:
```

```
        if i != j:
```

```
            model.addConstr(
```

```
                u[i] - u[j] + (N-1) * x[i, j] <= N-2,
```

```
                name=f"mtz_{i}_{j}"
```

```
            )
```

```
# -----
```

```
# Cash/inventory constraints
```

```
# -----
```

```
# Truck capacity.
```

```
model.addConstr(
```

```
    gp.quicksum(q[i] for i in customers) <= TRUCK_CAPACITY_LAKH,
```

```
    name="truck_cash_capacity"
```

```
)
```

```
# Cash can only be delivered if ATM is visited, and cannot exceed ATM capacity.
```

```
for i in customers:
```

```
    max_delivery = float(df.iloc[i-1]["max_delivery_lakh"])
```

```
    model.addConstr(
```

```

    q[i] <= max_delivery * y[i],

    name=f"delivery_visit_{i}"

)

# Scenario shortage:

# current cash + delivery + shortage >= scenario demand.

for i in customers:

    current = float(df.iloc[i-1]["current_cash_lakh"])

    for scenario in SCENARIOS:

        demand = float(df.iloc[i-1][scenario])

        model.addConstr(

            current + q[i] + shortage[i, scenario] >= demand,

            name=f"shortage_{i}_{scenario}"

        )

# -----

# Route duration

# -----

travel_time = gp.quicksum(

    (dist[i, j] / AVERAGE_SPEED_KMPH) * x[i, j]

    for i, j in dist

)

service_time = gp.quicksum(

    float(df.iloc[i-1]["service_hours"]) * y[i]

    for i in customers

```

```
)
```

```
model.addConstr(
```

```
    travel_time + service_time <= MAX_ROUTE_HOURS,
```

```
    name="route_duration"
```

```
)
```

```
# -----
```

```
# Time windows
```

```
# -----
```

```
BIG_M = 50.0
```

```
# If the truck travels  $i \rightarrow j$ , arrival at  $j$  must occur after:
```

```
# arrival at  $i$  + service time at  $i$  + travel time.
```

```
for i in customers:
```

```
    for j in customers:
```

```
        if  $i \neq j$ :
```

```
            model.addConstr(
```

```
                 $t[j] \geq$ 
```

```
                 $t[i]$ 
```

```
                + float(df.iloc[i-1]["service_hours"])
```

```
                + dist[i, j] / AVERAGE_SPEED_KMPH
```

```
                - BIG_M * (1 -  $x[i, j]$ ),
```

```
                name=f"time_{i}_{j}"
```

```
            )
```

```
# Depot departure occurs at 9:00.
```

```

for j in customers:

    model.addConstr(

        t[j] >=

            9.0

            + dist[0, j] / AVERAGE_SPEED_KMPH

            - BIG_M * (1 - x[0, j]),

        name=f"depot_time_{j}"

    )

```

Below is the objective function.

```

travel_cost = TRAVEL_COST_PER_KM * gp.quicksum(

    dist[i, j] * x[i, j] for i, j in dist

)

```

```

visit_cost = VISIT_FIXED_COST * gp.quicksum(

    y[i] for i in customers

)

```

```

holding_cost = CASH_HOLDING_COST * gp.quicksum(

    q[i] for i in customers

)

```

```

expected_shortage_cost = gp.quicksum(

    SHORTAGE_PENALTY * probability * shortage[i, scenario]

    for scenario, probability in SCENARIOS.items()

    for i in customers

)

```

```

model.setObjective(
travel_cost + visit_cost + holding_cost + expected_shortage_cost,
GRB.MINIMIZE
)

```

Data Generation

The dataset contains ATM coordinates, current cash, expected demand, ATM capacity, priority, and service time. The normal-demand values are the central forecast. Low demand is 80% of the forecast and high demand is 130% of the forecast.

ATM	Location	Current ₹L	Normal Demand ₹L	Capacity ₹L	Priority	Low/Normal/High	Service h
A01	Salt Lake Sector V	1.8	3.2	6.0	3	2.6/3.2/4.2	0.20
A02	New Town	1.2	4.0	7.0	2	3.2/4.0/5.2	0.22
A03	Ultadanga	1.5	2.5	5.0	2	2.0/2.5/3.2	0.18
A04	Dum Dum	1.0	3.0	6.0	2	2.4/3.0/3.9	0.18
A05	Howrah	1.4	3.8	6.0	3	3.0/3.8/4.9	0.20
A06	Esplanade	1.0	4.5	7.0	3	3.6/4.5/5.9	0.20
A07	Park Street	1.3	3.0	5.0	3	2.4/3.0/3.9	0.18
A08	Ballygunge	1.0	3.5	6.0	2	2.8/3.5/4.5	0.18
A09	Gariahat	0.9	4.2	7.0	3	3.4/4.2/5.5	0.20
A10	Jadavpur	1.1	3.0	5.0	2	2.4/3.0/3.9	0.18
A11	Behala	1.3	2.8	5.0	1	2.2/2.8/3.6	0.16
A12	Tollygunge	1.0	3.2	6.0	2	2.6/3.2/4.2	0.18
A13	Kasba	1.2	2.6	5.0	2	2.1/2.6/3.4	0.18
A14	EM Bypass	1.1	4.0	7.0	3	3.2/4.0/5.2	0.20
A15	Science City	1.4	2.4	4.0	1	1.9/2.4/3.1	0.16

Results

The important result is not simply the shortest route. The model should be interpreted as a resource-allocation decision. Because the truck can carry only ₹25 lakh and at most ten ATMs can be visited, the optimizer must prioritize locations. Priority-3 ATMs are protected by hard constraints, while other ATMs compete for truck capacity and route time.

Under a high-demand scenario, shortages may remain even after optimization because the truck cannot carry enough cash to completely cover all possible demand. The shortage penalty determines how aggressively the model concentrates cash on important or high-risk ATMs.

The output on running the python program is as below:

===== *MODEL RESULTS* =====

Objective value: 413.98

ATMs visited: 10

Route:

0. Cash Depot

1. Salt Lake Sector V

2. New Town

3. EM Bypass

4. Jadavpur

5. Tollygunge

6. Gariahat

7. Ballygunge

8. Park Street

9. Howrah

10. Esplanade

11. Cash Depot

Travel distance: 73.60 km

Cash loaded: 25.00 lakh

Route duration: 4.88 hours

Cash allocation:

A01 Salt Lake Sector V: 1.40 lakh

A02 New Town: 2.80 lakh

A05 Howrah: 2.80 lakh

A06 Esplanade: 3.50 lakh

A07 Park Street: 1.70 lakh

A08 Ballygunge: 2.50 lakh

A09 Gariahat: 3.30 lakh

A10 Jadavpur: 1.90 lakh

A12 Tollygunge: 2.20 lakh

A14 EM Bypass: 2.90 lakh

Scenario shortages:

low_demand_lakh: 4.24 lakh

normal_demand_lakh: 6.90 lakh

high_demand_lakh: 21.41 lakh

Conclusions

The proposed model is a substantially richer OR formulation than a simple TSP. It integrates routing, inventory allocation, capacity constraints, demand uncertainty, priority, and scheduling within a single MILP framework. The project also provides a natural path toward more advanced topics such as stochastic programming, vehicle routing, and robust optimization.